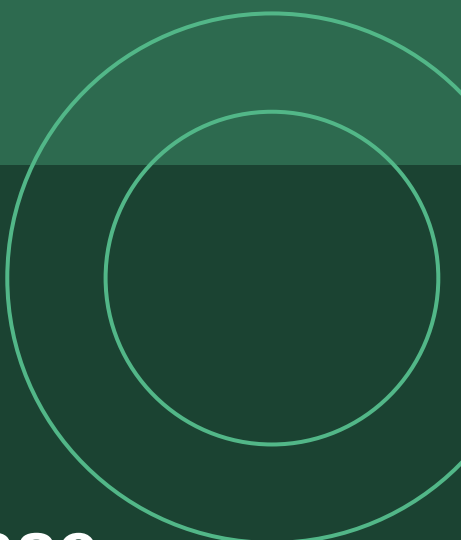




ProtoPNet Crop Disease

Research Pipeline Documentation

Stage 2 — Data Preprocessing Pipeline

Full Technical Reference

PlantWild v2 & Crop Pest Dataset

1. Overview

What preprocessing does and why it matters

Preprocessing is Stage 2 of the full ProtoPNet pipeline. Its purpose is to transform raw, unorganised image downloads into a clean, standardised, and correctly split dataset that can be fed directly into a PyTorch DataLoader without any further manipulation.

The quality of preprocessing directly determines the quality of learned prototypes. ProtoPNet works by finding real image patches that best represent each disease class. If the input images are blurry, duplicated, or inconsistently sized, the prototype push step in Phase 3 of training will select low-quality patches, producing uninterpretable explanations.

***Note:** The same preprocessing script is used for BOTH PlantWild and Crop Pest datasets with identical settings. This ensures any performance difference between the two datasets is attributable to the model and training procedure — not inconsistent data handling.*

Pipeline at a Glance

Step	Operation	Purpose	Removes Images?
1	Dataset Scan	Discover all image paths grouped by class	No
2	Deduplication	Remove byte-identical images (MD5 hash)	Yes
3	Blur Filtering	Remove blurry images via Laplacian variance	Yes
4	Class Balance Check	Warn on severe imbalance, save distribution chart	No
5	Stratified Split	Divide into 70% train / 15% val / 15% test	No
6	Resize & Copy	Write 224x224 JPEGs to output folder structure	No
7	Summary Report	Write per-class counts and config to .txt file	No

2. Input Format

How the script reads your raw dataset

The script expects the raw dataset to be organised into one subfolder per class, all living inside a single root directory. This is the standard format produced by most Kaggle dataset downloads.

Required Folder Structure

```
RAW_DATA_DIR/  
  
  Tomato_Early_Blight/  
  
    img001.jpg  
  
    img002.jpg  
  
    img003.png  
  
  Corn_Healthy/  
  
    img004.jpg  
  
    img005.jpg  
  
  Apple_Scab/  
  
    img006.jpg  
  
  ... (one folder per class)
```

The subfolder name becomes the class label. There is no CSV, no JSON metadata file, and no annotation file required. The script discovers classes purely from the directory structure.

Supported Image Formats

Extension	Format	Notes
.jpg / .jpeg	JPEG	Most common — fully supported
.png	PNG	Supports transparency (RGBA) — converted to RGB
.bmp	Bitmap	Uncompressed — supported, large file sizes
.tiff	TIFF	High quality — supported
.webp	WebP	Modern web format — supported

CSV-Based Datasets (Alternative Input)

If your dataset uses a flat image folder with a CSV label file instead of class subfolders (common in some Kaggle formats), a helper function `reorganise_from_csv()` is included at the bottom of the script. Run it first to reorganise into the required structure, then point `RAW_DATA_DIR` at the output.

```
# Only needed if your dataset is CSV-based (not class folders):  
reorganise_from_csv(  
    csv_path    = "labels.csv",  
    image_root = "./raw_images_flat",  
    output_root= "./plantwild_raw",  
    img_col     = "filename",  
    label_col   = "label"  
)
```

3. Step-by-Step Processing

What happens to your images and why

1

Dataset Scan

Walk the raw directory tree and collect all image file paths, grouped by class name.

The script uses Python's `pathlib` to walk every subdirectory in `RAW_DATA_DIR`. For each subdirectory it collects all files whose extension matches the supported formats. The result is a Python dictionary held in memory:

```
{
    "Tomato_Early_Blight": [Path("img001.jpg"), Path("img002.jpg"), ...],
    "Corn_Healthy":       [Path("img004.jpg"), ...],
    "Apple_Scab":         [Path("img006.jpg"), ...],
    ...
}
```

No files are copied or moved at this stage. This is purely a discovery and indexing pass. `tqdm` displays a progress bar across class directories. The total image count and class count are logged immediately after.

2

Deduplication

Remove exact duplicate images using MD5 content hashing across the entire dataset.

Duplicate images are a real problem in large scraped datasets like PlantWild. They cause data leakage between train and test splits if the same image lands in both. MD5 hashing detects duplicates by file content rather than filename — two files with different names but identical bytes will be caught.

How it works:

- For every image, read the file in 64KB chunks and feed into an MD5 hasher.
- Store the resulting hash in a `seen_hashes` dictionary mapping hash → first file path.
- If a new file produces a hash already in the dictionary → it is a duplicate → skip it.
- If the hash is new → add to the dictionary and keep the file.
- The class → files dictionary is updated to exclude all duplicates.

Note: Deduplication operates across the entire dataset, not just within a class. An image that appears in two different class folders (a mislabelled duplicate) will also be caught.

3

Blur Filtering

Remove images that are too blurry or too small to be useful training examples.

PlantWild is an in-the-wild dataset — images were taken in real field conditions with phones, not laboratory cameras. A significant proportion will be motion-blurred, out-of-focus, or otherwise low quality. Including these harms prototype quality because the prototype push step may select a blurry patch as the

representative of a disease class.

Two checks are applied to every image:

Check 1 — File size: If the file is smaller than `MIN_FILE_SIZE_KB` (default: 1 KB) it is almost certainly corrupt or empty. Removed immediately without opening the image.

Check 2 — Laplacian variance: The image is read in grayscale using OpenCV. A Laplacian operator (second-order edge-detection filter) is applied to the pixel intensities. The variance of the resulting matrix is computed. Sharp images with clear edges produce high variance. Blurry images produce low variance.

```
# Laplacian variance calculation (core logic)

img = cv2.imread(filepath, cv2.IMREAD_GRAYSCALE)

score = cv2.Laplacian(img, cv2.CV_64F).var()

# score < BLUR_THRESHOLD → image is blurry → remove
```

BLUR_THRESHOLD	Effect	Recommended for
50	Very lenient — only extreme blur removed	High-quality datasets
100	Default — good balance for PlantWild	PlantWild, most field datasets
200	Strict — removes mildly blurry images	If prototype quality is poor

4

Class Balance Check

Audit the cleaned dataset for severe class imbalance before splitting.

Class imbalance is a known challenge in plant disease datasets. PlantWild has 89 classes ranging from common diseases with thousands of samples to rare conditions with only dozens. This step does not remove or resample any images — it is a diagnostic and warning system.

What it produces:

- A logged table of all classes sorted by image count, with ASCII bar lengths for quick visual comparison.
- A warning if `max_class_count / min_class_count` exceeds `IMBALANCE_RATIO_WARN` (default: 10x).
- A bar chart PNG saved to the output directory showing the full distribution.

Note: If severe imbalance is detected, consider using weighted cross-entropy loss during training by passing `class_weights` to `torch.nn.CrossEntropyLoss`. The preprocessing script does not apply oversampling or undersampling — that is handled at the `DataLoader` level.

5

Stratified Split

Divide the cleaned dataset into train, validation, and test sets while maintaining class proportions.

A naive random split risks placing all examples of a rare class into the training set, leaving none for evaluation. Stratified splitting solves this by ensuring each class contributes proportionally to all three splits.

Split ratios:

Split	Ratio	Purpose
train	70%	Used for gradient updates, prototype push, and FC fine-tuning
val	15%	Used for hyperparameter tuning and early stopping decisions
test	15%	Held-out final evaluation — never seen during training

How it works technically:

scikit-learn's StratifiedShuffleSplit does not support three-way splitting directly. The split is performed in two passes:

```
Pass 1: all data → train (70%) + val_test (30%)
Pass 2: val_test → val (50% of 30% = 15%) + test (50% of 30% = 15%)

Result: train=70%, val=15%, test=15% — all stratified.
```

The output is three Python lists, each containing (file_path, class_name) tuples. These lists are passed directly to the resize and copy step.

6

Resize & Copy to Output

The only step that writes to disk — resizes each image to 224x224 and saves to the output folder structure.

This step reads each source image, applies the size transformation, and writes the result to the correct location in the output directory. The raw source files are never modified or deleted — everything is written as new files.

Processing applied to each image:

Operation	Detail	Reason
Open	Pillow Image.open()	Handles all supported formats
Convert	img.convert('RGB')	Forces 3 channels — handles RGBA PNG and grayscale
Resize	img.resize((224,224), LANCZOS)	Standard size for DenseNet161 backbone
Save	JPEG at quality=95	Consistent format, minimal quality loss

Output structure written to disk:

```
OUTPUT_DIR/  
  train/  
    Tomato_Early_Blight/  
      img001.jpg  
      img002.jpg  
    Corn_Healthy/  
      img004.jpg  
    ...  
  val/  
    Tomato_Early_Blight/  
      img008.jpg  
    ...  
  test/  
    Tomato_Early_Blight/  
      img009.jpg  
    ...
```

Note: This exact structure is what PyTorch's `torchvision.datasets.ImageFolder` expects. You can point your `DataLoader` directly at `OUTPUT_DIR/train/` with no extra configuration.

7**Summary Report**

Write a full preprocessing record to disk — class counts per split, config used, and totals.

After all images are written, the script generates a plain-text summary file at `OUTPUT_DIR/preprocessing_summary.txt`. This file records exactly how many images each class contributed to each split, which is useful for verifying the stratified split worked correctly and for the dataset statistics section of your paper.


```
=====

PREPROCESSING SUMMARY

=====

Dataset root      : ./data/plantwild_raw
Output dir       : ./data/plantwild
Image size       : (224, 224)
Blur threshold   : 100.0
Random seed      : 42


Total classes    : 89
Total clean      : 17,841
  Train         : 12,489 (70.0%)
  Val           :  2,676 (15.0%)
  Test          :  2,676 (15.0%)


Per-class counts (train | val | test):
  Tomato_Early_Blight      840 | 180 | 180
  Corn_Healthy              623 | 134 | 134
  ...

=====
```

4. Configuration Reference

All settings at the top of the script

All configurable values live in the CONFIG dictionary at the top of preprocess.py. Change these before running — do not hardcode values elsewhere in the script.

Setting	Default	Description
RAW_DATA_DIR	./data/plantwild_raw	Path to your raw downloaded dataset
OUTPUT_DIR	./data/plantwild	Where cleaned, split data will be written
IMG_SIZE	(224, 224)	Resize target — must match backbone input size
BLUR_THRESHOLD	100.0	Laplacian variance cutoff — raise to be stricter
MIN_FILE_SIZE_KB	1	Files smaller than this are treated as corrupt
TRAIN_RATIO	0.70	Fraction of clean data used for training
VAL_RATIO	0.15	Fraction used for validation
TEST_RATIO	0.15	Fraction used for testing (must sum to 1.0)
IMBALANCE_RATIO_WARN	10	Warn if max/min class count exceeds this
IMG_EXTENSIONS	{.jpg, .jpeg, .png, .bmp, ...}	File types to include during scan
RANDOM_SEED	42	Seed for reproducible splits
LOG_FILE	preprocessing.log	Path where full log is written
PLOT_CLASS_DIST	True	Whether to save class distribution bar chart

Reusing for Crop Pest Dataset

To preprocess the Crop Pest dataset, simply change two settings and re-run:

```
CONFIG = {  
    "RAW_DATA_DIR": "./data/cropest_raw",    # Change this  
    "OUTPUT_DIR":   "./data/cropest",        # Change this  
    # All other settings remain identical to PlantWild run  
    ...  
}
```

Note: Using identical settings for both datasets is intentional. Any difference in model performance between PlantWild and Crop Pest can then be attributed to the data content and model design, not differences in preprocessing.

5. Output Files

Everything the script produces

File / Folder	Location	Contents
train/	OUTPUT_DIR/	70% of clean images, class subfolders, 224x224 JPEG
val/	OUTPUT_DIR/	15% of clean images, class subfolders, 224x224 JPEG
test/	OUTPUT_DIR/	15% of clean images, class subfolders, 224x224 JPEG
class_distribution.png	OUTPUT_DIR/	Bar chart of per-class image counts after cleaning
preprocessing_summary.txt	OUTPUT_DIR/	Per-class train/val/test counts + config settings
preprocessing.log	Script run dir	Full log: every warning, skip, duplicate, blur fail

Using the Output in PyTorch

The output folder is directly compatible with `torchvision.datasets.ImageFolder`. Here is the standard `DataLoader` setup for the training stage:

```
from torchvision import datasets, transforms

from torch.utils.data import DataLoader

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406], # ImageNet mean
        std =[0.229, 0.224, 0.225]  # ImageNet std
    )
])

train_dataset = datasets.ImageFolder("./data/plantwild/train", transform=transform)
val_dataset   = datasets.ImageFolder("./data/plantwild/val",   transform=transform)
test_dataset  = datasets.ImageFolder("./data/plantwild/test",  transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=4)
val_loader   = DataLoader(val_dataset,   batch_size=64, shuffle=False, num_workers=4)
test_loader  = DataLoader(test_dataset,  batch_size=64, shuffle=False, num_workers=4)
```

Note: Pixel normalisation (ImageNet mean/std) is intentionally NOT applied during preprocessing. It is applied in the DataLoader transform at training time. This keeps the saved images in standard viewable format and makes the preprocessing script dataset-agnostic.

6. Required Libraries

Install before running

```
pip install Pillow tqdm numpy scikit-learn opencv-python matplotlib
```

Library	Version	Used for
Pillow	>=9.0	Opening, converting, resizing, and saving images
tqdm	>=4.0	Progress bars on all long-running loops
numpy	>=1.21	Array operations for stratified split indexing
scikit-learn	>=1.0	StratifiedShuffleSplit for reproducible splits
opencv-python	>=4.5	Laplacian variance blur detection
matplotlib	>=3.4	Class distribution bar chart generation

7. Next Steps

What comes after preprocessing is complete

1

Verify the output

Check preprocessing_summary.txt. Confirm total counts match expectations. Review class_distribution.png for any severe imbalance that needs addressing.

2

Set up augmentation pipeline

Stage 2b (not yet implemented) adds training augmentations: random flips, rotation, colour jitter, random crop, and Gaussian blur. These are applied in the DataLoader transform — not to the saved files.

3

Build the DataLoader

Use torchvision.datasets.ImageFolder pointing at OUTPUT_DIR/train/, val/, and test/. Apply ImageNet normalisation in the transform. This is covered in Stage 3.

4

Begin Phase 1 Training

With clean data loaders in place, proceed to Stage 3: ProtoPNet architecture setup and Phase 1 warm-up training on PlantWild.